12 Objetos y clases

Objetos y clases

La Programación Orientada a Objetos es un **paradigma de programación que se basa en el concepto de "objetos"**.

Estos son agrupaciones que **representan entidades del mundo real** con sus propias características (atributos) y comportamientos (métodos).

En POO, los objetos interactúan entre sí a través de mensajes, lo que permite modelar de manera más natural los sistemas complejos.

Definición de una clase en Python

En Python, las clases se definen con la palabra clave **class** seguida del nombre de la clase y dos puntos (:).

Una clase es una plantilla o un "molde" para crear objetos. Define las propiedades y comportamientos que tendrán los objetos que se creen a partir de ella.

```
class Persona:
    def __init__(self, nombre, edad):
    self.nombre = nombre
    self.edad = edad

    def saludar(self):
    print(f"Hola, mi nombre es {self.nombre} y tengo {self.edad}
años.")
```

El método **__init__** es un método especial llamado "constructor", que se ejecuta automáticamente al crear un nuevo objeto de la clase. Aquí inicializamos los atributos nombre y edad.

12 Objetos y clases

El método saludar es un método de instancia que muestra un saludo con el nombre y edad de la persona.

- **Atributos**: Los atributos son variables que pertenecen a un objeto y que representan sus características o propiedades. Pueden ser de cualquier tipo de dato (como números, cadenas, listas, etc). En el ejemplo anterior, nombre y edad son atributos del objeto persona.

- **Métodos**: Los métodos son funciones que pertenecen a un objeto y que pueden operar sobre sus atributos (pueden realizar operaciones como modificar los valores de los atributos, o interactuar con otros objetos). En el ejemplo, saludar es un método del objeto persona.

Creación de instancias

Una **instancia** es **cada uno de los "ejemplares" que creamos a partir de una clase**. Cada instancia tendrá sus tiene sus propios valores para los atributos definidos en la clase.

Para crear un objeto a partir de una clase, simplemente **se llama al nombre de la clase seguido de paréntesis**.

Por ejemplo, una vez que tenemos la clase Persona, podemos crear instancias de esta clase así,

```
luis = Persona("Luis", 30)
```

En este caso, hemos creado un objeto luis a partir de la clase Persona, con los valores "Luis" para el atributo nombre y 30 para el atributo edad.

12 Objetos y clases

Acceso a atributo y métodos

Para acceder tanto a los atributos o métodos de una instancia, simplemente usamos el operador de acceso . sobre la variable que contiene la instancia.

Por ejemplo así,

```
luis = Persona("Luis", 30)
ana = Persona("Ana", 25)

luis.edad = 35;  # Cambiamos edad de Luis a 35
ana.edad = 28;   # Cambiamos edad de ana a 28

luis.saludar()  # Salida: Hola, mi nombre es Luis y tengo 35 años.
ana.saludar()   # Salida: Hola, mi nombre es Ana y tengo 28 años.
```

12 Objetos y clases

Constructores

Un constructor en Python es un método especial **__init__()** que se llama automáticamente cuando se crea una nueva instancia de una clase.

Su función principal es inicializar los atributos de la instancia con valores predeterminados o personalizados.s

Ejemplo básico de constructor

```
# Definición de una clase con constructor
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

# Creación de instancias utilizando el constructor
persona1 = Persona("Luis", 30)
persona2 = Persona("María", 25)

# Acceso a los atributos de las instancias
print(persona1.nombre, persona1.edad)  # Salida: Luis 30
print(persona2.nombre, persona2.edad)  # Salida: María 25
```

En este ejemplo,

- **__init__(self, nombre, edad)** es el constructor de la clase Persona
- Cuando se crea una nueva instancia (**persona1** y **persona2**)
- Python automáticamente llama a __init__() y pasa los argumentos nombre y edad proporcionados.

Parámetros del constructor

El primer parámetro de **__init__()** es self, que representa la instancia actual de la clase. Se utiliza para acceder a los atributos y métodos de la instancia dentro del propio método y en otros métodos de la clase.

12 Objetos y clases

Además de self, el constructor puede aceptar cualquier número de parámetros necesarios para inicializar la instancia. Estos parámetros son proporcionados al crear la instancia.

Definición de atributos opcionales

Los constructores pueden definir atributos opcionales con valores predeterminados.

```
class Coche:
    def __init__(self, marca, modelo, color="negro"):
        self.marca = marca
        self.modelo = modelo
        self.color = color

# Creación de instancias
coche1 = Coche("Toyota", "Corolla")
coche2 = Coche("Tesla", "Model S", "rojo")

print(coche1.marca, coche1.modelo, coche1.color)  # Salida: Toyota
Corolla negro
print(coche2.marca, coche2.modelo, coche2.color)  # Salida: Tesla
Model S rojo
```

En este ejemplo, **Coche** tiene un atributo opcional **color** con valor predeterminado **"negro"**. Se puede proporcionar un color diferente al crear la instancia (**coche2**).

Constructor predeterminado

Si no se define un constructor (__init__()), Python proporciona uno por defecto que no inicializa ningún atributo.

12 Objetos y clases

Tipos de métodos

En Python, los métodos son funciones incluidas dentro de la definición de una clase.

Existen tres tipos de métodos, que se diferencian en cómo se definen y a qué tienen acceso:

- **Método de instancia**: Se utiliza cuando el método necesita acceder o modificar datos específicos de una instancia

- **Método de clase**: Se utiliza cuando el método necesita acceder a datos que pertenecen a toda la clase, como atributos de clase compartidos

- **Método estático**: Se utiliza cuando el método no depende de datos de instancia o clase y realiza una operación independiente

Métodos de instancia

Los **métodos de instancia** son métodos que **actúan sobre las instancias de una clase**. Tienen acceso a los atributos de esas instancias a través del parámetro **self**.

Son los métodos más comunes en Python y se definen dentro de una clase utilizando **def**.

12 Objetos y clases

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        return f"Hola, soy {self.nombre} y tengo {self.edad}
años."

# Creación de instancia de persona
persona1 = Persona("Luis", 30)

# Llamada al método de instancia
print(persona1.saludar())  # Salida: Hola, soy Luis y tengo 30
años.
```

En este ejemplo

- **saludar()** es un método de instancia
- Utiliza **self** para acceder a los atributos **nombre** y **edad** de la instancia **persona1**.

Métodos de clase

Los métodos de clase son métodos que **actúan sobre la clase misma** (en lugar de las instancias individuales de la clase).

Se definen utilizando el decorador **@classmethod** y tienen acceso a la clase a través del parámetro **cls**.

```
class Coche:
    descuento = 0.1  # Descuento del 10% para todos los coches

    def __init__(self, marca, modelo, precio):
        self.marca = marca
```

```
        self.modelo = modelo
        self.precio = precio

    @classmethod
    def aplica_descuento(cls, precio):
        return precio * (1 - cls.descuento)

# Uso del método de clase
precio_final = Coche.aplica_descuento(30000)
print(f"Precio final con descuento: ${precio_final}")  # Salida:
Precio final con descuento: $27000.0
```

En este ejemplo,

- aplica_descuento() es un método de clase
- Utiliza cls para acceder al atributo de clase descuento y aplicar un descuento al precio proporcionado.

Métodos estáticos

Los métodos estáticos son métodos que **se relacionan con la clase pero no requieren acceso a sus atributos.**

Se definen **utilizando el decorador @staticmethod**, y no dependen de la instancia (**self**) ni de la clase (**cls**).

```
class Utilidades:
    @staticmethod
    def sumar(a, b):
        return a + b

# Uso del método estático
resultado = Utilidades.sumar(3, 5)
print(f"Resultado de la suma: {resultado}")  # Salida:
Resultado de la suma: 8
```

12 Objetos y clases

En este ejemplo,

- **sumar()** es un método estático que realiza una operación simple de suma sin necesidad de acceder a atributos de instancia o clase.

12 Objetos y clases

Modificadores de acceso

La **encapsulación** en Python es un concepto que se refiere a la **restricción del acceso directo a algunos componentes de un objeto,** con el objetivo de prevenir modificaciones accidentales y asegurar un uso controlado de sus atributos y métodos.

Python **no implementa el encapsulamiento de manera "estricta"** como otros lenguajes orientados a objetos. Pero si proporciona mecanismos que permiten gestionar la visibilidad y acceso a los miembros de una clase.

Para ello, Python **utiliza convenciones de nombres** para indicar la accesibilidad de los miembros de una clase.

Acceso público

Los atributos y métodos que no comienzan con un guion bajo (_) se consideran públicos y pueden ser accedidos desde fuera de la clase.

```
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre  # Atributo público

    def saludar(self):
        return f"Hola, soy {self.nombre}"  # Método público

# Uso de la clase Persona
persona = Persona("Luis")
print(persona.saludar())  # Salida: Hola, soy Luis
```

12 Objetos y clases

Acceso protegido

Los atributos y métodos que **comienzan con un solo guion bajo (_) se consideran protegidos** y no deben ser accedidos directamente desde fuera de la clase.

```
class CuentaBancaria:
    def __init__(self, saldo):
        self._saldo = saldo  # Atributo protegido

    def mostrar_saldo(self):
        return f"Saldo disponible: {self._saldo}"  # Método
protegido

# Uso de la clase CuentaBancaria
cuenta = CuentaBancaria(5000)
print(cuenta.mostrar_saldo())  # Salida: Saldo disponible: 5000
print(cuenta._saldo)  # Acceso protegido (no recomendado pero
posible)
```

Acceso privado

Los atributos y métodos que **comienzan con dos guiones bajos (__) se consideran privados** y no deben ser accedidos directamente desde fuera de la clase.

```
class Estudiante:
    def __init__(self, nombre):
        self.__nombre = nombre  # Atributo privado

    def __presentarse(self):
        return f"¡Hola! Soy {self.__nombre}"  # Método privado
```

12 Objetos y clases

```
# Uso de la clase Estudiante
estudiante = Estudiante("María")
# print(estudiante.__nombre)  # Error: AttributeError (no se puede
acceder directamente)
# estudiante.__presentarse()  # Error: AttributeError (no se puede
llamar directamente)
```

No obstante, realmente no tenemos una encapsulación estricta. Hay formas de acceder a los campos y métodos, incluso aunque estén marcados como “privados” con __.

Es decir, están “bastante protegidos”, pero no 100% protegidos. Aunque también os digo, que saltaros la protección casi seguro que sea siempre una mala idea .

12 Objetos y clases

Métodos Dunder

En Python, los métodos Dunder (abreviatura de “double underscore”) son aquellos cuyo nombre comienza y termina con dos guiones bajos (__).

Estos métodos no se llaman directamente, sino que son invocados automáticamente por el intérprete de Python en diversas situaciones (como operaciones aritméticas, manipulación de secuencias y gestión del contexto).

También conocidos como métodos mágicos o especiales

Algunos de los métodos dunder más comunes son:

Método	Descripción
__init__	Inicializa una nueva instancia de una clase
__str__	Devuelve una string de un objeto, amigable para el usuario
__repr__	Devuelve una string de un objeto, amigable para el desarrollador
__len__	Devuelve la longitud de un objeto
__getitem__	Permite acceder a elementos mediante índices
__setitem__	Permite asignar valores a elementos mediante índices
__delitem__	Permite eliminar elementos mediante índices
__iter__	Devuelve un iterador para el objeto
__next__	Devuelve el siguiente elemento del iterador

Implementación de Métodos Dunder

Veamos cómo se implementan y utilizan algunos de estos métodos en una clase en Python.

Método __init__

El método __init__ se utiliza para inicializar los atributos de una clase cuando se crea una nueva instancia.

```
class Persona:
    def __init__(self, nombre, edad):
```

```
        self.nombre = nombre
        self.edad = edad

persona1 = Persona("Alice", 30)
print(persona1.nombre)  # Salida: Alice
print(persona1.edad)    # Salida: 30
```

Métodos __str__ y __repr__

Los métodos __str__ y __repr__ devuelven representaciones en cadena de un objeto. La diferencia principal es que __str__ está destinado a una representación amigable para el usuario, mientras que __repr__ está orientado a los desarrolladores y debe ser más detallado.

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def __str__(self):
        return f"Persona: {self.nombre}, {self.edad} años"

    def __repr__(self):
        return f"Persona('{self.nombre}', {self.edad})"

persona1 = Persona("Alice", 30)
print(str(persona1))  # Salida: Persona: Alice, 30 años
print(repr(persona1)) # Salida: Persona('Alice', 30)
```

Método __len__

El método __len__ se utiliza para devolver la longitud de un objeto.

12 Objetos y clases

```
class Grupo:
    def __init__(self, miembros):
        self.miembros = miembros

    def __len__(self):
        return len(self.miembros)

grupo = Grupo(["Alice", "Bob", "Charlie"])
print(len(grupo))  # Salida: 3
```

Métodos __getitem__, __setitem__ y __delitem__

Estos métodos permiten que los objetos de una clase se comporten como contenedores (listas, diccionarios, etc.).

```
class MiLista:
    def __init__(self):
        self.data = []

    def __getitem__(self, index):
        return self.data[index]

    def __setitem__(self, index, value):
        self.data[index] = value

    def __delitem__(self, index):
        del self.data[index]

mi_lista = MiLista()
mi_lista.data = [1, 2, 3, 4, 5]
print(mi_lista[2])   # Salida: 3
mi_lista[2] = 30
print(mi_lista[2])   # Salida: 30
del mi_lista[2]
print(mi_lista.data) # Salida: [1, 2, 4, 5]
```

12 Objetos y clases

Métodos __iter__ y __next__

Estos métodos permiten que los objetos de una clase sean iterables.

```
class Contador:
    def __init__(self, max):
        self.max = max
        self.contador = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.contador < self.max:
            self.contador += 1
            return self.contador
        else:
            raise StopIteration

contador = Contador(5)
for numero in contador:
    print(numero)
```

Métodos Dunder para operaciones aritméticas

Además de los métodos mencionados, Python permite sobrecargar operadores aritméticos usando métodos dunder como __add__, __sub__, __mul__, __truediv__, entre otros.

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, otro):
        return Vector(self.x + otro.x, self.y + otro.y)
```

12 Objetos y clases

```
    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(2, 3)
v2 = Vector(4, 5)
v3 = v1 + v2
print(v3)  # Salida: Vector(6, 8)
```

12 Objetos y clases

Herencia

La herencia es otro concepto importante en POO, que nos permite **crear nuevas clases, basadas en otras clases existentes.**

La nueva clase hereda los atributos y métodos de la clase base y puede agregar sus propios atributos y métodos. De esta forma, la subclase puede "extender" o modificar el comportamiento de la superclase.

Así, llamamos:

- **Superclase**: La clase de la cual se heredan los métodos y atributos.
- **Subclase**: La clase que hereda métodos y atributos de la superclase

En Python, para hacer que una clase herede de otra **añadimos el nombre de la super clase entre paréntesis** justo tras la definición de la clase. Así

```
class subclase(superclase):
   # cuerpo de subclase
```

Ejemplo de herencia

Vamos a verlo con una serie de ejemplos. En primer lugar, supongamos que tenemos una clase **Estudiante** que hereda de **Persona.**

```
# Definición de la clase Persona
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
```

```
    def mostrar_informacion(self):
        print(f"Nombre: {self.nombre}, Edad: {self.edad}")
```

```
# Definición de la clase Estudiante que hereda de Persona
class Estudiante(Persona):
    def __init__(self, nombre, edad, nota):
        super().__init__(nombre, edad)
        self.nota = nota

# Crear una instancia de Estudiante
estudiante = Estudiante("Luis", 20, "8.7")
estudiante.mostrar_informacion()   # imprime 'Nombre: Luis, Edad:
20'
```

En este ejemplo,

- **Estudiante** hereda de **Persona** usando **class Estudiante(Persona)**
- La función **super()** se utiliza para llamar al constructor de la clase base **Persona**, lo que permite inicializar **nombre** y **edad**.

Sobreescritura de métodos

En este ejemplo, **Estudiante** sobreescribe el método **mostrar_informacion** de **Persona**.

```
# Definición de la clase Estudiante que hereda de Persona
class Estudiante(Persona):
    def __init__(self, nombre, edad, nota):
        super().__init__(nombre, edad)
        self.nota = nota

    # Sobreescribir el método mostrar_informacion
```

```
    def mostrar_informacion(self):
        print(f"Nombre: {self.nombre}, Edad: {self.edad}, Nota:
{self.nota}")

# Crear una instancia de Estudiante
estudiante = Estudiante("Luis", 20, "8.7")
estudiante.mostrar_informacion()   # imprime 'Nombre: Luis, Edad:
20, Nota: 8.7'
```

Aquí, **Estudiante** sobreescribe el método **mostrar_informacion** de **Persona** para incluir información adicional sobre la matrícula. Esto se logra definiendo un nuevo método **mostrar_informacion** en la clase **Estudiante**.

Llamar a métodos de la superclase

La clase derivada puede llamar a métodos de su clase base usando el método super(). De hecho, es habitual hacerlo con el método constructor __init__ para inicializar los atributos de la clase base.

Pero **cualquier método de la clase base puede ser invocado**. Vamos a verlo con un ejemplo.

```
# Definición de la clase Estudiante que hereda de Persona
class Estudiante(Persona):
    def __init__(self, nombre, edad, nota):
        super().__init__(nombre, edad)  # Llamada al constructor
de la clase base
        self.nota = nota

    def mostrar_informacion(self):
        super().mostrar_informacion(self)  # Llamada a un método
de la clase base
        print(f"Nota: {self.nota}")
```

```
# Crear una instancia de Estudiante
estudiante = Estudiante("Luis", 20, "8.7") # imprime 'Nombre:
Luis, Edad: 20, Nota: 8.7'
estudiante.mostrar_informacion()
```

En este ejemplo,

- **super().__init__(nombre, edad)** invoca el constructor de Persona, para inicializar los campos nombre y edad.
- **super().mostrar_informacion(self)** invoca el al método **mostrar_informacion** de **Persona**, y luego añade más acciones (en el ejemplo, un **print**)

Herencia múltiple

Python soporta la herencia múltiple (lo que significa que una subclase puede heredar de múltiples superclases).

Esto se logra listando las superclases entre paréntesis, después del nombre de la subclase.

```
class A:
    def metodo_a(self):
        print("Método de clase A")

class B:
    def metodo_b(self):
        print("Método de clase B")

class C(A, B):
    def metodo_c(self):
        print("Método de clase C")

# Creación de instancia de C y uso de métodos
objeto_c = C()
objeto_c.metodo_a()  # Salida: Método de clase A
objeto_c.metodo_b()  # Salida: Método de clase B
objeto_c.metodo_c()  # Salida: Método de clase C
```

12 Objetos y clases

En este ejemplo, la clase C hereda tanto de A como de B, permitiendo el uso de métodos definidos en ambas superclases.

12 Objetos y clases

Polimorfismo

El polimorfismo es la capacidad de un objeto para tomar diferentes formas según el contexto.

En Python, esto se logra a través del “duck typing”, lo que básicamente significa que el funcionamiento de un objeto se determina por su comportamiento, en lugar de su tipo específico.

Vamos a verlo con un ejemplo. Supongamos que tenemos nuestra clase Persona.

```
# Definición de la clase Persona
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        print(f"Nombre: {self.nombre}, Edad: {self.edad}")
```

Y una o varias clases derivadas

```
# Definición de la clase Profesor que hereda de Persona
class Profesor(Persona):
    def __init__(self, nombre, edad):
        super().__init__(nombre, edad)

    def saludar(self):
        print("Soy un profesor")

# Definición de la clase Estudiante que hereda de Persona
class Estudiante(Persona):
    def __init__(self, nombre, edad):
        super().__init__(nombre, edad)

    def saludar(self):
```

```
        print("Soy un estudiante")
```

Por otro lado, supongamos que tenemos una función que recibe un parámetro

```
# Función que usa polimorfismo
def imprimir_informacion(persona):
    persona.saludar()
```

Ahora, creamos dos instancias de **Profesor** y **Estudiante** respectivamente, y se lo pasamos al método **imprimir_informacion.**

```
# Crear instancias de Persona y Estudiante
profesor = Profesor("María", 30)
estudiante = Estudiante("Luis", 20)

# Usar polimorfismo
imprimir_informacion(profesor)     # Imprime 'Soy un profesor'
imprimir_informacion(estudiante)  # Imprime 'Soy un estudiante'
```

En este ejemplo,

- La función **imprimir_informacion** puede aceptar objetos tanto de la clase **Persona** como de la clase **Estudiante**, o **Profesor**
- La función llama al método **saludar** de cada objeto

Es decir, básicamente a Python "le da igual" el tipo del objeto que le pasas. Si tiene los métodos, **la función llamará al método que tenga el objeto.**

Le da igual su tipo, o si heredan de una clase o o no. Sólo le interesa si tiene los métodos que necesita.

12 Objetos y clases

Uso de Getters y Setters

Los métodos getters y setters son funciones que **permiten obtener y establecer el valor de un atributo de una clase,** respectivamente.

Se utilizan para **agregar lógica adicional al acceso de los atributos,** como validación o transformación de datos.

Implementación de Getters y Setters

Los métodos getters y setters se definen utilizando métodos normales dentro de la clase.

```
class Persona:
    def __init__(self, nombre, edad):
        self._nombre = nombre
        self._edad = edad

    def get_nombre(self):
        return self._nombre

    def set_nombre(self, nombre):
        if isinstance(nombre, str):
            self._nombre = nombre
        else:
            raise ValueError("El nombre debe ser una cadena de
texto")

    def get_edad(self):
        return self._edad

    def set_edad(self, edad):
        if isinstance(edad, int) and edad > 0:
            self._edad = edad
        else:
```

```
            raise ValueError("La edad debe ser un número entero
positivo")
```

Por lo demás, no tienen nada más en especial. Son simplemente métodos que tienen la convención de llamarse **get_** y **set_** (con el nombre de la propiedad detrás) y cuya función es obtener o establecer el valor de un atributo.

Uso de Getters y Setters

Para utilizar estos métodos, simplemente tenemos que llamar explícitamente a los métodos get_ y set_.

```
persona = Persona("Luis", 25)
print(persona.get_nombre())  # Salida: Luis
persona.set_nombre("Carlos")
print(persona.get_nombre())  # Salida: Carlos

print(persona.get_edad())  # Salida: 25
persona.set_edad(30)
print(persona.get_edad())  # Salida: 30
```

12 Objetos y clases

Como usar Propiedades

Python proporciona una forma más elegante y concisa de definir getters y setters **utilizando la función property**.

Esta función permite definir una propiedad que **se usa como un atributo, pero con la capacidad de controlar su acceso y modificación**.

Las propiedades hacen que el acceso a los atributos parezca más natural, y su sintaxis es más limpia y fácil de entender.

Definición de propiedades

Se utiliza la palabra reservada property para definir una propiedad. Se puede pasar opcionalmente un getter, un setter y un deleter a esta función.

```
class Persona:
    def __init__(self, nombre, edad):
        self._nombre = nombre
        self._edad = edad

    @property
    def nombre(self):
        return self._nombre

    @nombre.setter
    def nombre(self, nombre):
        if isinstance(nombre, str):
            self._nombre = nombre
        else:
            raise ValueError("El nombre debe ser una cadena de
texto")

    @property
    def edad(self):
        return self._edad
```

```
    @edad.setter
    def edad(self, edad):
        if isinstance(edad, int) and edad > 0:
            self._edad = edad
        else:
            raise ValueError("La edad debe ser un número entero
positivo")
```

Uso de propiedades

Con las propiedades definidas, se pueden obtener y establecer valores utilizando la sintaxis de atributos, lo que hace el código más limpio y legible.

```
persona = Persona("Luis", 25)
print(persona.nombre)  # Salida: Luis
persona.nombre = "Carlos"
print(persona.nombre)  # Salida: Carlos

print(persona.edad)  # Salida: 25
persona.edad = 30
print(persona.edad)  # Salida: 30
```

Fijaos que no tengo que invocar a un método setter o getter. Simplemente obtengo el valor con ., y lo escribo con =, igual que si fuera un atributo.

12 Objetos y clases

Clases Abstractas

Una **clase abstracta** es **una clase que no se puede instanciar directamente**. Su propósito principal es ser heredada por otras clases que implementan los métodos definidos como abstractos en la clase base.

En Python, las clases abstractas se implementan utilizando el módulo abc (Abstract Base Classes). Este módulo proporciona herramientas para definir clases y métodos abstractos.

```
from abc import ABC, abstractmethod
```

Definiendo una Clase Abstracta

Para definir una clase abstracta, se debe heredar de ABC y utilizar el decorador @abstractmethod para marcar los métodos que deben ser implementados por las subclases.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def hacer_sonido(self):
        pass

    def dormir(self):
        print("El animal está durmiendo.")
```

En este ejemplo,

- **Animal** es una clase abstracta con un método abstracto **hacer_sonido**
- Un método concreto **dormir**.

Implementando Clases Derivadas

Las clases que heredan de una clase abstracta **deben implementar todos los métodos abstractos**. Si una subclase no implementa un método abstracto, también se convierte en una clase abstracta y no se puede instanciar.

Veamos como implementar nuestra clase abstracta **Animal**

```
class Perro(Animal):
    def hacer_sonido(self):
        print("El perro ladra.")

class Gato(Animal):
    def hacer_sonido(self):
        print("El gato maúlla.")
```

En este ejemplo, Perro y Gato son subclases de Animal que implementan el método abstracto hacer_sonido.

Clases mixtas

Una clase abstracta puede tener tanto métodos abstractos como métodos concretos. Los métodos concretos proporcionan una implementación por defecto que puede ser heredada o sobrescrita por las subclases.

Vamos a verlo con un ejemplo,

12 Objetos y clases

```python
class Vehiculo(ABC):
    @abstractmethod
    def conducir(self):
        pass

    def encender(self):
        print("El vehículo está encendido.")

class Coche(Vehiculo):
    def conducir(self):
        print("El coche está conduciendo.")

class Moto(Vehiculo):
    def conducir(self):
        print("La moto está conduciendo.")
```

En este ejemplo,

- **Vehículo** tiene un método abstracto **conducir** y un método concreto **encender**.
- Las subclases **Coche** y **Moto** implementan **conducir** pero heredan **encender**.

Resumen comparativo

Característica	Clases Abstractas	Clases Mixtas (Mixins)
Propósito	Definir un contrato y una plantilla de comportamiento.	Reutilizar funcionalidad con implementación.
Métodos abstractos	Sí, es su característica principal.	No.
Métodos concretos	Sí.	Sí, y es su función principal.
Instanciación	No permitida.	Generalmente no, pero posible.
Obligación en la clase hija	Implementar los métodos abstractos.	No hay una obligación, se añaden capacidades.